

Project & Backend

Interview Q&A

Tailored to Sushant Sonawane's Resume — CodeSync · Sush.Eats · Gemini Chatbot · Node/Express Backend

PROJECTS → BACKEND → DATABASES → SECURITY → DEPLOYMENT → SYSTEM DESIGN

This set is built specifically from your resume's projects, skills, and internships — no overlap with the general JavaScript or React notes. Expect interviewers to dig into "why," "how would you scale it," and "what broke."

Table of Contents

- | | |
|---|---|
| 1. 1. CodeSync — Real-Time Code Editor | 7. 7. Databases — MongoDB & SQL |
| 2. 2. Sush.Eats — Food Ordering App | 8. 8. Authentication, Security & Middleware |
| 3. 3. Gemini AI Chatbot Clone | 9. 9. Git, Deployment & Tools |
| 4. 4. Node.js & Express.js Backend | 10. 10. System Design & Scalability |
| 5. 5. REST API Design | 11. 11. Behavioral & Resume Wrap-Up |
| 6. 6. Socket.IO & Real-Time Communication | |

01 CodeSync — Real-Time Code Editor

This is your flagship project — expect it to get the most interview time. Be ready to draw the architecture on a whiteboard.

Q: Walk me through how CodeSync works end to end.

A: CodeSync is a real-time collaborative code editor. A user creates or joins a "room" using a unique session ID. The React frontend connects to the Node.js/Express backend over a Socket.IO connection. Every keystroke or code change is emitted as a socket event, broadcast to everyone else in that same room, and applied to their editor instantly — giving the effect of multiple people typing in the same file simultaneously.

Likely follow-up: "What's in the payload of a code-change event?" — Be ready to describe it: room ID, the changed content/delta, and the sender's socket ID (so you don't echo the change back to the person who made it).

Q: Why did you choose Socket.IO over plain WebSockets?

A: Socket.IO sits on top of WebSockets but adds automatic reconnection, fallback to HTTP long-polling if WebSockets aren't available, built-in support for "rooms" and "namespaces," and easy event-based communication — all things I'd otherwise have to build manually with raw WebSockets.

Q: How does room-based collaboration work technically?

A: When a user joins with a room/session ID, the server calls `socket.join(roomId)`. From then on, the server broadcasts events with `io.to(roomId).emit(...)`, so only sockets in that specific room receive the update — keeping different coding sessions completely isolated from each other.

```
socket.on("join-room", (roomId, userName) => {
  socket.join(roomId);
  socket.to(roomId).emit("user-joined", userName);
});

socket.on("code-change", ({ roomId, code }) => {
  socket.to(roomId).emit("code-update", code); // broadcast to others in room only
});
```

Q: What happens if two users edit the same line at the exact same time?

A: In the current version, it's essentially "last write wins" — whichever change reaches the server last overwrites the shared state, since there's no operational transform (OT) or CRDT-based conflict resolution yet. That's an honest limitation and a great "what would you improve" answer.

Good answer if pushed further: "For true concurrent-edit safety like Google Docs, I'd look into a CRDT library like Yjs or Automerge, which merge concurrent edits deterministically without a central lock."

Q: How would you scale CodeSync to handle thousands of concurrent rooms?

A: Single Node instance holds all Socket.IO connections in memory, which doesn't scale horizontally by default — if you add a second server, users in the same room could land on different servers and never see each other's updates. The fix is the Socket.IO Redis adapter, which lets multiple Node instances share room/broadcast state through Redis pub/sub, plus a load balancer with "sticky sessions" so a client's polling fallback requests keep hitting the same server.

Q: How do you handle a user disconnecting mid-session?

A: Socket.IO fires a `disconnect` event automatically; the server can then notify remaining room members ("X left the room") and optionally clean up empty rooms after the last user leaves to free memory.

Q: What was the trickiest bug you fixed in this project?

A: I hit a bug where the MongoDB schema was missing the `userName` field, so joined users weren't being saved/retrieved correctly — a good reminder that Mongoose schemas need to match every field actually being sent from the frontend. I also debugged socket event handling issues on the frontend where listeners weren't being cleaned up properly, and a 403 error from the Gemini API caused by an invalid/expired key, which taught me to double check environment variable loading order with dotenv.

02 Sush.Eats — Food Ordering App

Frontend-only project — interviewers will often ask you to reason about the backend it doesn't have yet.

Q: This project is frontend-only — how did you simulate a backend, and what would you add if it needed one?

A: For the frontend build, menu/category data is handled as local/mock data structures rendered dynamically. If I added a real backend, I'd build a Node/Express API with MongoDB collections for menu items, orders, and users, with endpoints like `GET /api/menu`, `POST /api/orders`, and JWT-based auth for checkout.

Q: How did you structure routing between Home, Menu, Cart, and Checkout?

A: I used React Router with each page as its own route/component, so navigation between them doesn't trigger a full page reload — just swaps out the matched component while shared layout (like a header/cart icon) stays mounted.

Q: You mention "prop drilling minimization" on your resume — what did you actually do?

A: I kept shared state (like the cart) as high as needed but grouped related components so data only had to pass through one or two levels instead of many, and used component composition so presentational components didn't need to know about cart logic at all — only the components that actually needed cart data received it.

Likely follow-up: "Would Context API have helped here?" — Yes, especially for cart state accessed from the header, cart page, and checkout — a good sign you understand when Context is the right call versus when it's overkill.

Q: How would you persist the cart across a page refresh?

A: Store the cart state in `localStorage` alongside React state — read it once on mount to initialize state, and write to it whenever the cart changes, so a refresh doesn't wipe out the user's selections.

Q: If asked to add real checkout/payment, how would you approach it?

A: I'd integrate a payment gateway (like Razorpay or Stripe) on the backend to keep secret keys off the client, create an order record in MongoDB before payment, and use a webhook from the payment provider to confirm success and update the order status — never trusting a client-side "payment successful" flag alone.

03 Gemini AI Chatbot Clone

Q: How does your chatbot maintain conversation flow?

A: Messages (both user and bot) are stored in a state array, each rendered as a message bubble component. Every new user message is appended to that array, sent to the Gemini API, and the response is appended back once it arrives — `useRef` is used to auto-scroll to the latest message and manage input focus after sending.

Q: Where does the Gemini API call happen — client-side or via your backend? Why does that matter?

A: In my Node/Express backend integration work, I moved the Gemini API call server-side specifically so the API key stays in an environment variable on the server and is never exposed in the browser's network tab — calling it directly from React would leak the key to anyone who opens dev tools.

Q: How do you handle errors or rate limits from the Gemini API?

A: Wrap the API call in try/catch, check the response status (I ran into a 403 from an invalid key during development), and show a friendly fallback message in the chat UI instead of letting the whole request silently fail with no feedback to the user.

Q: What modes does your AI assist controller support, and how did you structure that on the backend?

A: The backend AI assist controller supports explain, fix, improve, and freeform modes — each mode sends a differently-crafted prompt to `gemini-1.5-flash` based on which mode the frontend requests, so the same endpoint can serve multiple use cases just by varying the prompt template server-side.

04 Node.js & Express.js Backend

Q: Why is Node.js a good fit for a real-time app like CodeSync?

A: Node uses a single-threaded, event-driven, non-blocking I/O model — it can handle thousands of simultaneous open connections (like Socket.IO sockets) efficiently without spinning up a new thread per connection, which is exactly the workload a real-time collaborative app needs.

Q: Explain the Express middleware pattern.

A: Middleware are functions that run in sequence for each incoming request, each with access to `req`, `res`, and a `next()` function to pass control to the next one. They're used for things like parsing JSON bodies, logging, authentication checks, and error handling — each middleware can either respond directly or hand off to the next.

```
function authMiddleware(req, res, next) {
  const token = req.headers.authorization;
  if (!token) return res.status(401).json({ error: "No token" });
  // verify token...
  next(); // pass control forward
}
app.use("/api/protected", authMiddleware);
```

Q: What's the difference between `app.use()` and `app.get()/app.post()`?

A: `app.use()` registers middleware that runs for any HTTP method matching the given path prefix (or all paths if none is given). `app.get()` / `app.post()` register handlers for one specific HTTP method and exact route.

Q: How do you structure a scalable Express project?

A: I separate concerns into folders: `routes/` (URL → controller mapping), `controllers/` (request handling logic), `models/` (Mongoose schemas), `middleware/` (auth, error handling), and `config/` (env setup, DB connection) — this keeps route files thin and logic testable in isolation.

05 REST API Design

Q: What makes an API "RESTful"?

A: It follows a resource-oriented design (URLs represent nouns, like `/api/users`), uses standard HTTP methods to represent actions (GET/POST/PUT/DELETE), is stateless (each request carries all the info needed, no server-side session dependency), and returns proper status codes and structured responses (usually JSON).

Q: What status codes would you use for a typical CRUD flow?

A: 200 OK for a successful GET/PUT, 201 Created for a successful POST that creates a resource, 204 No Content for a successful DELETE, 400 Bad Request for invalid input, 401 Unauthorized for missing/invalid auth, 404 Not Found for a missing resource, and 500 for unexpected server errors.

Q: What is idempotency, and which HTTP methods are idempotent?

A: An idempotent request produces the same result no matter how many times it's repeated. GET, PUT, and DELETE are idempotent (calling PUT twice with the same data leaves the resource in the same state); POST is NOT idempotent, since calling it twice typically creates two separate resources.

Q: How would you version an API?

A: Most commonly via the URL path (`/api/v1/users` , `/api/v2/users`), which is simple and explicit — alternatives include custom headers or query parameters, but URL versioning is the easiest for clients to understand and for routing to handle cleanly.

06 Socket.IO & Real-Time Communication

Q: How is Socket.IO different from raw WebSockets?

A: Raw WebSockets give you just the low-level bidirectional connection — you'd have to build reconnection logic, room management, and fallback transport yourself. Socket.IO wraps WebSockets with all of that built in: automatic reconnection, "rooms," "namespaces" for logically separating traffic, and graceful fallback to HTTP long-polling if a WebSocket connection can't be established.

Q: What's the difference between a "room" and a "namespace" in Socket.IO?

A: A namespace is a separate communication channel at the top level (e.g. `/chat` vs `/admin`), typically set up once at connection time. A room is a more dynamic, lightweight grouping WITHIN a namespace that sockets can join/leave at any time — exactly what I used for each CodeSync session.

Q: How does Socket.IO handle reconnection if a user's WiFi drops briefly?

A: The client automatically attempts to reconnect with an increasing backoff delay by default. On the server, you typically need to re-join the socket to its room after reconnecting (since a new connection means a new socket ID), which is why I store the room ID client-side and re-emit "join-room" on the `connect` event.

Q: How do you broadcast a message to everyone in a room EXCEPT the sender?

A: Use `socket.to(roomId).emit(...)` from within that socket's own handler — this broadcasts to everyone in the room except the socket that called it. Using `io.to(roomId).emit(...)` instead would send it to everyone in the room including the sender.

07 Databases — MongoDB & SQL

Q: Why MongoDB for a project like CodeSync rather than a SQL database?

A: MongoDB's flexible, document-based schema fits fast-changing/loosely-structured data well (like session metadata, chat messages, or user profiles that may evolve), and it pairs naturally with a Node/Express stack via Mongoose. For CodeSync specifically, session/room data doesn't need complex relational joins, which is where SQL would shine instead.

Q: Explain how a missing field in your Mongoose schema caused a real bug.

A: I had a `userName` field being sent from the frontend on join, but it wasn't declared in the Mongoose schema — by default, Mongoose silently drops fields not defined in the schema (unless `strict: false` is set), so the value was never actually saved, even though no error was thrown. Adding the field to the schema fixed it immediately.

```
const roomSchema = new mongoose.Schema({
  roomId: String,
  userName: String, // was missing - caused silent data loss
  joinedAt: { type: Date, default: Date.now }
});
```

Q: SQL vs NoSQL — when would you choose one over the other?

A: Choose SQL (like PostgreSQL/MySQL) when data is highly relational and needs strict consistency and complex joins — think orders, payments, inventory with strong referential integrity. Choose NoSQL (MongoDB) when the schema is likely to evolve, data is naturally document-shaped, or you need to scale horizontally with high write throughput.

Q: What is database indexing and why does it matter?

A: An index is a data structure (commonly a B-tree) that lets the database find matching rows/documents without scanning the entire collection/table — critical for query performance as data grows, though every index adds some write overhead and storage cost, so you index fields you actually query/filter/sort on frequently.

08 Authentication, Security & Middleware

Q: How does JWT-based authentication work end to end?

A: On login, the server verifies credentials and signs a JWT containing the user's ID (and maybe role) using a secret key, sending it back to the client. The client stores it and sends it in the **Authorization** header on future requests. The server's auth middleware verifies the token's signature and expiry on each protected request, without needing to hit the database to check a session — that's what makes JWT auth stateless.

```
const jwt = require("jsonwebtoken");
function verifyToken(req, res, next) {
  try {
    const token = req.headers.authorization?.split(" ")[1];
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (err) {
    res.status(401).json({ error: "Invalid token" });
  }
}
```

Q: Where should you store a JWT on the client, and what are the tradeoffs?

A: **localStorage** is easy but vulnerable to XSS (any injected script can read it). An httpOnly cookie is safer against XSS since JavaScript can't read it, but needs CSRF protection instead. For most of my projects I used localStorage for simplicity, but I'd move to httpOnly cookies for anything handling sensitive data in production.

Q: What is CORS and why does it come up constantly in MERN apps?

A: CORS (Cross-Origin Resource Sharing) is a browser security feature that blocks a frontend on one origin (e.g. **localhost:5173**) from calling an API on a different origin (e.g. **localhost:5000**) unless the server explicitly allows it via response headers — solved in Express with the **cors** middleware package, configured to allow your specific frontend domain(s).

Q: How do you keep a third-party API key (like the Gemini key) secure?

A: Store it in a **.env** file (never committed to Git), load it with **dotenv**, and only ever call the third-party API from the backend — the frontend calls YOUR backend endpoint, and your backend attaches the real key server-side, so it's never exposed in browser network requests.

09 Git, Deployment & Tools

Q: Walk me through your typical Git workflow on a project.

A: I work on feature branches off `main` (e.g. `feature/socket-rooms`), commit in small logical chunks with clear messages, push and open a pull request, then merge back into `main` once it's working — keeping `main` always in a deployable state.

Q: How do you resolve a merge conflict?

A: Git marks the conflicting sections in the file with `<<<<<<< / ===== / >>>>>>>` markers — I review both versions, manually decide (or combine) the correct final code, remove the markers, then stage and commit the resolved file.

Q: Why use Netlify/Vercel for frontend but Render for backend?

A: Netlify and Vercel are optimized for static/frontend builds (React apps) with fast global CDN delivery and instant deploys on every git push. Render is better suited for long-running backend processes (Node/Express servers with persistent Socket.IO connections) that need a continuously running server rather than a static build.

Q: How do you manage environment variables safely across local and deployed environments?

A: Locally, a `.env` file (added to `.gitignore`) holds secrets loaded via `dotenv`. On the deployment platform (Render/Vercel), the same variables are set through their dashboard's environment variable settings, so the same code works locally and in production without hardcoding secrets anywhere in the codebase.

Q: How do you use Postman before wiring an API up to the frontend?

A: I test each endpoint in isolation — checking request/response shape, status codes, and edge cases (missing fields, invalid auth) — before touching the frontend, so I know for certain whether a bug is coming from the backend or the frontend integration when something goes wrong later.

10 System Design & Scalability

Your resume says you're "expanding expertise in backend development and system design" — expect at least one basic system design question.

Q: How would you scale CodeSync's backend across multiple servers?

A: Add the Socket.IO Redis adapter so multiple Node instances can share broadcast/room state through Redis pub/sub, put a load balancer in front with sticky sessions (so polling fallback clients keep hitting the same instance), and keep MongoDB as a separate, independently scalable service.

Q: What's the difference between horizontal and vertical scaling?

A: Vertical scaling means giving a single server more resources (CPU/RAM) — simple but has a hard ceiling. Horizontal scaling means adding more servers to share the load — more complex (needs load balancing and shared state) but scales much further and adds redundancy.

Q: How would you design a basic caching layer to reduce database load?

A: Put Redis in front of frequently-read, rarely-changed data (like menu items in Sush.Eats) — check the cache first, only hit MongoDB on a cache miss, and set a reasonable expiry (TTL) so stale data doesn't linger too long after an update.

Q: How would you monitor a production backend for issues?

A: Basic logging (request logs, error logs) is the starting point, then tools like a hosted logging/monitoring service to track uptime, response times, and error rates — for a small project, even simple console logging plus Render's built-in dashboard metrics is a reasonable starting point before reaching for something heavier.

11 Behavioral & Resume Wrap-Up

Q: Tell me about a challenging bug you fixed and how you approached debugging it.

A: (Use the CodeSync socket/schema bugs as your story) — I noticed data wasn't persisting correctly, checked the network tab to confirm the frontend WAS sending the right payload, then traced it to the backend where the Mongoose schema silently dropped the unrecognized field. That taught me to always log/verify what's actually reaching the database, not just assume the schema matches the payload.

Q: Why are you focusing more on backend and system design now, coming from a frontend-heavy background?

A: Building CodeSync exposed me to real backend challenges — handling concurrent connections, data consistency, and scaling — and I realized being a strong full-stack developer means understanding both sides well enough to make informed architecture decisions, not just consume APIs someone else designed.

Q: What would you improve if you rebuilt CodeSync from scratch today?

A: I'd add a CRDT library (like Yjs) for real conflict-free concurrent editing instead of last-write-wins, add the Redis adapter from day one for horizontal scaling, add proper auth so rooms aren't accessible to anyone with the link, and add automated tests around the socket event handlers since those were the hardest bugs to catch manually.

Q: How did your AI/ML internship inform how you approached the Gemini chatbot project?

A: The internship gave me a foundation in how models are trained and evaluated, which helped me reason about the Gemini API as a black-box model to prompt effectively — understanding things like why prompt structure affects the quality of the "explain/fix/improve" responses in my AI assist controller.

Tip: For every project answer above, be ready to open the actual GitHub repo and point at the specific file/line if asked "show me that in the code."